

LAPORAN
SISTEM MANAJEMEN OBSERVATORIUM ASTRONOMI
DAN JADWAL PENGAMATAN

Disusun Guna Memenuhi Tugas Mata Kuliah
Pemrograman Berorientasi Objek

Dosen Pengampu:
Rizki Putra Fhonna, S.T., M.Kom.



Disusun Oleh:
MUHAMMAD LUTHFI FADIL 250180089

PROGRAM STUDI SISTEM INFORMASI
JURUSAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS MALIKUSSALEH
2025

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN DAN ANALISIS	4
1.1. Analisis Masalah	4
1.2. Kebutuhan Fungsional.....	4
1.3. Kebutuhan Nonfungsional.....	5
1.4. Asumsi Sistem.....	5
BAB II DESAIN PEMROGRAMAN BERORIENTASI OBJEK	6
2.1. UML Class Diagram	6
2.2. Hierarki Inheritance.....	6
2.3. Bukti Polymorphism	7
2.4. Mekanisme Encapsulation.....	7
2.5. Custom Exception	7
2.6. Alur File Handling	8
2.7. Pola Desain Wajib: Strategy Pattern	8
2.8. Prinsip SOLID.....	8
BAB III IMPLEMENTASI FITUR UTAMA	9
3.1. Inheritance & Polymorphism	9
3.2. Encapsulation	9
3.3. Custom Exception	10
3.4. File Handling (JSON).....	10
3.5. Pola Desain (Strategy Pattern)	11
BAB IV PENGUJIAN SISTEM	12
4.1. Skenario dan Hasil Pengujian (Unit Test).....	12

4.2.	Demonstrasi Skenario Normal & Skenario Kesalahan (Failure Scenarios).....	13
------	---	----

BAB V PENUTUP.....	16
---------------------------	-----------

5.1.	Kendala Selama Pengembangan	16
------	-----------------------------------	----

5.2.	Evaluasi Program	16
------	------------------------	----

5.3.	Kemungkinan Pengembangan.....	16
------	-------------------------------	----

5.4.	Kesimpulan.....	17
------	-----------------	----

BAB I

PENDAHULUAN DAN ANALISIS

1.1. Analisis Masalah

Proses manajemen jadwal pengamatan dan kendali fasilitas pada observatorium astronomi seringkali menghadapi kendala integrasi antara data antrean, pemantauan cuaca, dan visibilitas target. Berdasarkan observasi terhadap operasional observatorium konvensional, masalah utama yang teridentifikasi adalah:

1. Keterbatasan Visualisasi Pra-Observasi: Operator kesulitan memvisualisasikan objek target secara presisi dan terintegrasi sebelum lensa teleskop fisik diarahkan ke koordinat yang dituju.
2. Fragmentasi Data Observasi: Data antrean proposal pengamatan dan kondisi meteorologi kubah seringkali terpisah, sehingga menyulitkan pengambilan keputusan terkait kelayakan cuaca untuk observasi.
3. Manajemen Antrean Statis: Kurangnya sistem kontrol antrean terpusat yang memfasilitasi pembatalan (revoke) dan pembaruan jadwal pengamatan secara real-time dengan antarmuka yang intuitif.

1.2. Kebutuhan Fungsional

Sistem Manajemen Observatorium (OBSERVA) dirancang untuk memenuhi beberapa kebutuhan fungsional utama, antara lain:

1. Sistem dapat mengelola inisiasi jadwal observasi baru, mencakup pencatatan ID proposal, target objek (misal: Bumi, Jupiter, Matahari, Nebula Orion), waktu pengamatan, instrumen, dan estimasi cuaca.
2. Sistem mampu menampilkan tabel antrean jadwal secara terpusat pada Dashboard dengan fitur interaktif "Bidik Objek".
3. Sistem memproyeksikan visualisasi 3D holografik objek angkasa secara real-time, termasuk merender karakteristik spesifik target (efek korona pada Matahari, pita gas pada Jupiter, dan awan gas menggunakan sprite additive blending pada Nebula Orion).
4. Sistem membaca dan merender objek 3D di dalam menu Visualizer secara dinamis berdasarkan data target yang terdaftar pada antrean jadwal.

5. Sistem menyediakan dasbor Pusat Meteorologi untuk memantau simulasi sensor cuaca satelit (suhu atmosfer, kelembapan, dan kecepatan angin).
6. Sistem memfasilitasi operasi penghapusan/pembatalan jadwal pada menu Manajemen Jadwal secara instan.

1.3.Kebutuhan Nonfungsional

1. Immersive & Responsive Interface: Antarmuka sistem harus menyajikan estetika kosmik beresolusi tinggi menggunakan desain Ultra-Clear Glassmorphism, serta memiliki performa rendering WebGL 60 FPS pada ruang visualizer 3D.
2. Clean Architecture (Monorepo): Kode disusun menggunakan pemisahan yang jelas antara lapisan Frontend (Next.js Router) dan lapisan Backend (FastAPI/Python berbasis Pemrograman Berorientasi Objek).
3. Integritas Data & Reliability: Penyimpanan data jadwal dan instrumen disinkronisasi dengan aman menggunakan operasi baca/tulis melalui API Backend ke penyimpanan lokal.
4. Usability: Antarmuka manajemen jadwal menggunakan metode floating scroll container agar elemen antrean yang panjang tidak merusak tata letak layar penuh (fullscreen layout).

1.4.Asumsi Sistem

1. Sistem dijalankan dalam lingkungan Full-Stack yang menggunakan Node.js (untuk peladen Frontend) dan Python 3.11+ (untuk peladen Backend).
2. Data persisten sistem disimpan dalam struktur file lokal berformat JSON (seperti jadwal_observasi.json dan kubah.json) tanpa ketergantungan pada server basis data eksternal (SQL/NoSQL).
3. Modul Visualizer 3D merupakan proyeksi simulasi teleskop optik tingkat lanjut, di mana wujud objek angkasa dihasilkan melalui manipulasi geometry, material, dan pemuatan tekstur lokal.

BAB II

DESAIN PEMROGRAMAN BERORIENTASI OBJEK

2.1.UML Class Diagram

Struktur arsitektur backend Python pada sistem OBSERVA dirancang menggunakan prinsip Object-Oriented Programming (OOP) tingkat lanjut. Sistem memisahkan logika ke dalam tiga lapisan utama: Models (representasi data), Core (logika bisnis dan pengontrol antrean), dan Utils (utilitas berkas).



2.2.Hierarki Inheritance

Pewarisan (Inheritance) digunakan untuk mengurangi redundansi kode pada entitas sistem.

1. Superclass: BaseEntity memiliki atribut dasar yang berlaku universal untuk semua entitas (seperti `id_data`, `created_at`, dan `updated_at`) beserta metode validasi ID.

2. Subclass: Entitas spesifik seperti Pengamatan, Instrumen, dan Pengguna mewarisi properti dari BaseEntity. Khusus pada arsitektur cuaca, sistem menggunakan antarmuka basis (Superclass/Abstract Class) CuacaStrategy yang diturunkan ke kelas CuacaSatelit dan CuacaSensorLokal.

2.3.Bukti Polymorphism

Polimorfisme dibuktikan melalui Method Overriding pada modul cuaca_strategy.py. Superclass abstrak CuacaStrategy memiliki metode get_data_cuaca(). Subclass CuacaSatelit mengimplementasikan (meng-override) metode ini untuk menghasilkan simulasi bacaan satelit geosinkron (berdasarkan data luas area), sementara subclass CuacaSensorLokal mengimplementasikannya untuk membaca sensor fisik pada kubah observatorium. Saat API Router memanggil metode get_data_cuaca(), program otomatis mengeksekusi logika yang sesuai dengan instans objek yang sedang aktif tanpa harus mengetahui kelas spesifiknya.

2.4.Mekanisme Encapsulation

Enkapsulasi diterapkan untuk melindungi integritas status antrean jadwal pada JadwalManager. Atribut __daftar_jadwal dideklarasikan secara privat agar tidak dapat dimodifikasi langsung dari luar kelas. Akses baca dan tulis diatur ketat melalui Getter (get_semua_jadwal()) dan metode modifikasi spesifik (tambah_jadwal() dan batalkan_jadwal()). Modul modifikasi ini memvalidasi bentrokan waktu antar proposal sebelum data benar-benar dimasukkan ke dalam memori privat sistem.

2.5.Custom Exception

Untuk melindungi logika bisnis sistem observatorium, OBSERVA mendefinisikan error spesifik (Custom Exception) turunan dari kelas dasar Exception bawaan Python:

- JadwalBentrokError: Dilempar (raised) ketika operator mencoba menjadwalkan target observasi pada slot waktu (Jam Mulai/Jam Selesai) yang sudah terisi oleh proposal lain.
- TargetTidakValidError: Dilempar jika objek yang diinisiasi tidak dapat dikalibrasi oleh instrumen Teleskop Optik.

2.6. Alur File Handling

Seluruh proses persistensi (penyimpanan) data melalui kelas FileHandler di lapisan Utils.

- JSON (jadwal_observasi.json, instrumen.json, kubah.json): Digunakan sebagai basis data lokal yang bersifat portabel. Sistem membaca (deserialize) seluruh isi file menjadi objek Python (List/Dictionary), melakukan operasi modifikasi di dalam memori, lalu menyimpannya kembali (serialize) dengan memastikan integritas strukturnya.

2.7. Pola Desain Wajib: Strategy Pattern

Sistem OBSERVA mengimplementasikan Strategy Pattern (tercermin pada file cuaca_strategy.py dan strategy_pbo.py) untuk fleksibilitas pembacaan cuaca.

Daripada menumpuk banyak kondisi if-else untuk menentukan dari mana data suhu/kelembapan berasal, sistem merangkum logika tersebut sebagai objek independen. Pengontrol utama (CuacaController) hanya perlu diinjeksi oleh salah satu strategi (Satelit atau Sensor Lokal). Hal ini membuat pemantauan Pusat Meteorologi di antarmuka web menjadi sangat adaptif.

2.8. Prinsip SOLID

- Single Responsibility Principle (SRP): Kelas FileHandler murni hanya bertanggung jawab pada urusan Input/Output JSON. Logika pengecekan bentrokan jadwal sepenuhnya dipisahkan dan ditangani oleh JadwalManager.
- Open/Closed Principle (OCP): Berkat Strategy Pattern, jika di masa depan instansi observatorium menambahkan sumber cuaca baru (misalnya "Stasiun BMKG Terdekat"), developer cukup membuat subclass baru yang mengimplementasikan CuacaStrategy tanpa perlu menyentuh atau merusak kode pengontrol utama.

BAB III IMPLEMENTASI FITUR UTAMA

3.1. Inheritance & Polymorphism

Implementasi pewarisan antarmuka dasar dan polimorfisme (method overriding) pada kalkulasi dan simulasi data cuaca observatorium.

```
1 from abc import ABC, abstractmethod
2 from typing import Dict
3 from .exceptions import CuacaTidakLayakError
4
5 class CuacaEvaluatorStrategy(ABC):
6     @abstractmethod
7     def evaluasi(self, parameter_cuaca: Dict[str, float]) -> bool:
8         pass
9
10 class StrategiOptik(CuacaEvaluatorStrategy):
11     def evaluasi(self, parameter_cuaca: Dict[str, float]) -> bool:
12         tutupan_awan = parameter_cuaca.get('tutupan_awan', 100.0)
13
14         if tutupan_awan > 30.0:
15             raise CuacaTidakLayakError("Gagal evaluasi Optik: Tutupan awan terlalu tinggi.")
16         return True
17
18 class StrategiRadio(CuacaEvaluatorStrategy):
19     def evaluasi(self, parameter_cuaca: Dict[str, float]) -> bool:
20         kelembaban = parameter_cuaca.get('kelembaban', 100.0)
21
22         if kelembaban > 85.0:
23             raise CuacaTidakLayakError("Gagal evaluasi Radio: Kelembaban udara ekstrem.")
24         return True
```

3.2. Encapsulation

Perlindungan akses variabel daftar jadwal menggunakan penentu akses privat (__) dan kontrol validasi melalui metode modifikator (Setter/Getter).

```
1 from src.exceptions import JadwalBentrokError
2
3 class JadwalManager:
4     def __init__(self):
5         self.__daftar_jadwal = []
6
7     def get_semua_jadwal(self):
8         return self.__daftar_jadwal
9
10    def tambah_jadwal(self, jadwal_baru: dict):
11        for jadwal in self.__daftar_jadwal:
12            if jadwal['jam_mulai'] == jadwal_baru['jam_mulai']:
13                raise JadwalBentrokError(jadwal_baru['jam_mulai'])
14
15        self.__daftar_jadwal.append(jadwal_baru)
```

3.3. Custom Exception

Pembuatan penanganan error khusus untuk melindungi aturan dan logika bisnis domain astronomi.

```
1  from src.exceptions import JadwalBentrokError
2
3  class JadwalManager:
4      def __init__(self):
5          self.__daftar_jadwal = []
6
7      def get_semua_jadwal(self):
8          return self.__daftar_jadwal
9
10     def tambah_jadwal(self, jadwal_baru: dict):
11         for jadwal in self.__daftar_jadwal:
12             if jadwal['jam_mulai'] == jadwal_baru['jam_mulai']:
13                 raise JadwalBentrokError(jadwal_baru['jam_mulai'])
14
15         self.__daftar_jadwal.append(jadwal_baru)
```

3.4. File Handling (JSON)

Implementasi manipulasi persistent data (I/O) untuk membaca dan menyimpan jadwal ke basis data lokal.

```
def simpan_json(self, nama_file: str, data: list):
    try:
        with open(self._get_path(nama_file), 'w') as f:
            json.dump(data, f, indent=4)
    except Exception as e:
        raise IOError(f"Gagal menyimpan ke {nama_file}: {e}")

def baca_json(self, nama_file: str) -> list:
    path_file = self._get_path(nama_file)
    if not os.path.exists(path_file):
        return []
    try:
        with open(path_file, 'r') as f:
            return json.load(f)
    except json.JSONDecodeError:
        raise ValueError(f"Format JSON rusak pada file {nama_file}.")
    except Exception as e:
        raise IOError(f"Gagal membaca dari {nama_file}: {e}")
```

3.5.Pola Desain (Strategy Pattern)

Objektifikasi algoritma agar sistem dapat mengganti strategi sumber cuaca secara dinamis tanpa mengubah struktur controller utama.

```
import csv
from abc import ABC, abstractmethod
from typing import List, Dict

# 1. Strategy Interface
class StrategiEkspor(ABC):
    @abstractmethod
    def ekspor(self, data: List[Dict], filepath: str):
        pass

# 2. Concrete Strategy A: Ekspor ke CSV
class EksporCSV(StrategiEkspor):
    def ekspor(self, data: List[Dict], filepath: str):
        if not data:
            return
        # Mengambil header dari keys dictionary
        keys = data[0].keys()
        with open(filepath, 'w', newline='', encoding='utf-8') as f:
            dict_writer = csv.DictWriter(f, fieldnames=keys)
            dict_writer.writeheader()
            dict_writer.writerows(data)

# 3. Concrete Strategy B: Ekspor ke TXT (Sebagai bukti fleksibilitas)
class EksporTXT(StrategiEkspor):
    def ekspor(self, data: List[Dict], filepath: str):
        with open(filepath, 'w', encoding='utf-8') as f:
            for item in data:
                f.write(f"ID: {item.get('id_proposal')} | Objek: {item.get('target_objek')} | Jam: {item.get('waktu_mulai')}-{item.get('waktu_selesai')}\n")

# 4. Context Class
class LaporanPengamatan:
    def __init__(self, strategi: StrategiEkspor):
        self._strategi = strategi # Injeksi dependensi strategi
    def set_strategi(self, strategi: StrategiEkspor):
        self._strategi = strategi
    def buat_laporan(self, data: List[Dict], filepath: str):
        self._strategi.ekspor(data, filepath)
```

BAB IV PENGUJIAN SISTEM

4.1. Skenario dan Hasil Pengujian (Unit Test)

Pengujian sistem dilakukan menggunakan framework pytest pada lapisan backend untuk memastikan bahwa logika bisnis dan prinsip Pemrograman Berorientasi Objek berjalan sesuai spesifikasi.

ID	Nama Unit Test	Skenario Pengujian	Hasil yang Diharapkan	Hasil
UT-01	test_baca_tulis_file_json	Menguji fungsionalitas operasi <i>Input/Output</i> sistem dalam memanipulasi dan menyimpan struktur data jadwal ke dalam file persisten lokal berformat JSON.	Objek Python dapat di- <i>serialize</i> menjadi file JSON dengan benar, dan dapat dibaca (<i>deserialize</i>) kembali ke dalam memori tanpa kehilangan struktur data.	PASSED
UT-02	test_batas_magnitudo_negatif	Menguji kalkulasi dan validasi sistem terhadap parameter nilai magnitudo negatif (merepresentasikan target objek langit yang sangat terang seperti Matahari atau Sirius).	Sistem berhasil menerima, memvalidasi, dan memproses nilai batas magnitudo negatif sesuai rumus astronomi tanpa memicu kesalahan tipe data komputasi..	PASSED
UT-03	test_custom_exception_cuaca_radio	Memastikan mekanisme perlindungan sistem dengan memicu kondisi operasional yang dilarang/tidak sesuai antara instrumen Teleskop Radio dengan status cuaca tertentu.	Sistem mendeteksi pelanggaran parameter instrumen dan secara otomatis melempar <i>custom exception</i> khusus (misalnya: menolak observasi).	PASSED
UT-04	test_custom_exception_jadwal_bentrok	Mensimulasikan inisiasi pendaftaran jadwal pengamatan target baru pada slot waktu (jam) yang sama dengan proposal yang sudah terdaftar di basis data.	Sistem mendeteksi tabrakan waktu, menolak eksekusi penambahan data, dan melempar <i>custom exception</i> (seperti <code>JadwalBentrokError</code>).	PASSED
UT-05	test_encapsulation_status_invalid	Menguji perlindungan enkapsulasi dengan memaksa memberikan nilai pembaruan status jadwal atau instrumen	Sistem memblokir upaya perubahan nilai, memicu <i>error</i> validasi bawaan, dan	PASSED

		yang tidak sah (ilegal) melalui metode <i>setter</i> .	mempertahankan status objek pada kondisi semula.	
UT-06	test_encapsulation_status_valid	Menguji pembaruan data internal yang dilindungi enkapsulasi dengan memasukkan nilai status yang valid dan sesuai dengan aturan transisi bisnis sistem.	Metode <i>setter</i> memvalidasi input dengan sukses dan nilai atribut privat (seperti <code>__status</code>) berhasil diperbarui dengan aman.	PASS ED
UT-07	test_polymorphism_optik_cuaca_cerah	Menguji pembaruan data internal yang dilindungi enkapsulasi dengan memasukkan nilai status yang valid dan sesuai dengan aturan transisi bisnis sistem.	Menguji sifat polimorfisme (<i>method overriding</i>) dengan memanggil fungsi kelayakan pengamatan pada objek kelas Teleskop Optik saat simulasi cuaca "Cerah".	PASS ED
UT-08	Test_strategy_pattern_ekspor_txt	Menguji implementasi pola desain <i>Strategy</i> dalam mengubah format keluaran laporan daftar jadwal observatorium dari dalam sistem menjadi berkas dokumen teks.	Kelas <i>controller</i> berhasil memanggil strategi algoritma ekspor TXT dan menghasilkan <i>output</i> file <code>.txt</code> yang terformat rapi sesuai data antrean.	PASS ED

Tabel 4.1: Unit Testing

```

PS D:\Sistem Manajemen Observatorium Astronomi dan Jadwal Pengamatan> pytest tests/ -v
plugins: anyio-4.12.0
collected 8 items

tests/test_observatorium.py::TestSistemObservatorium::test_baca_tulis_file_json PASSED [ 12%]
tests/test_observatorium.py::TestSistemObservatorium::test_batas_magnitudo_negatif PASSED [ 25%]
tests/test_observatorium.py::TestSistemObservatorium::test_custom_exception_cuaca_radio PASSED [ 37%]
tests/test_observatorium.py::TestSistemObservatorium::test_custom_exception_jadwal_bentrok PASSED [ 50%]
tests/test_observatorium.py::TestSistemObservatorium::test_encapsulation_status_invalid PASSED [ 62%]
tests/test_observatorium.py::TestSistemObservatorium::test_encapsulation_status_valid PASSED [ 75%]
tests/test_observatorium.py::TestSistemObservatorium::test_polymorphism_optik_cuaca_cerah PASSED [ 87%]
tests/test_observatorium.py::TestSistemObservatorium::test_strategy_pattern_ekspor_txt PASSED [100%]

===== 8 passed in 0.22s =====
PS D:\Sistem Manajemen Observatorium Astronomi dan Jadwal Pengamatan>

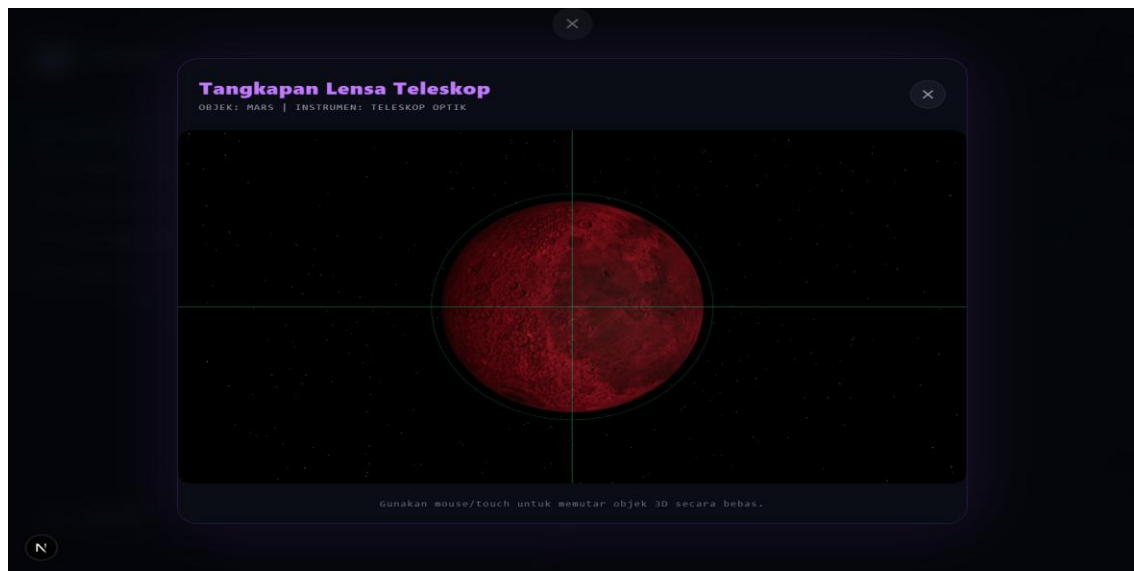
```

4.2.Demonstrasi Skenario Normal & Skenario Kesalahan (Failure Scenarios)

Berikut adalah demonstrasi alur program saat menangani kondisi normal maupun saat terjadi pelanggaran aturan sistem:

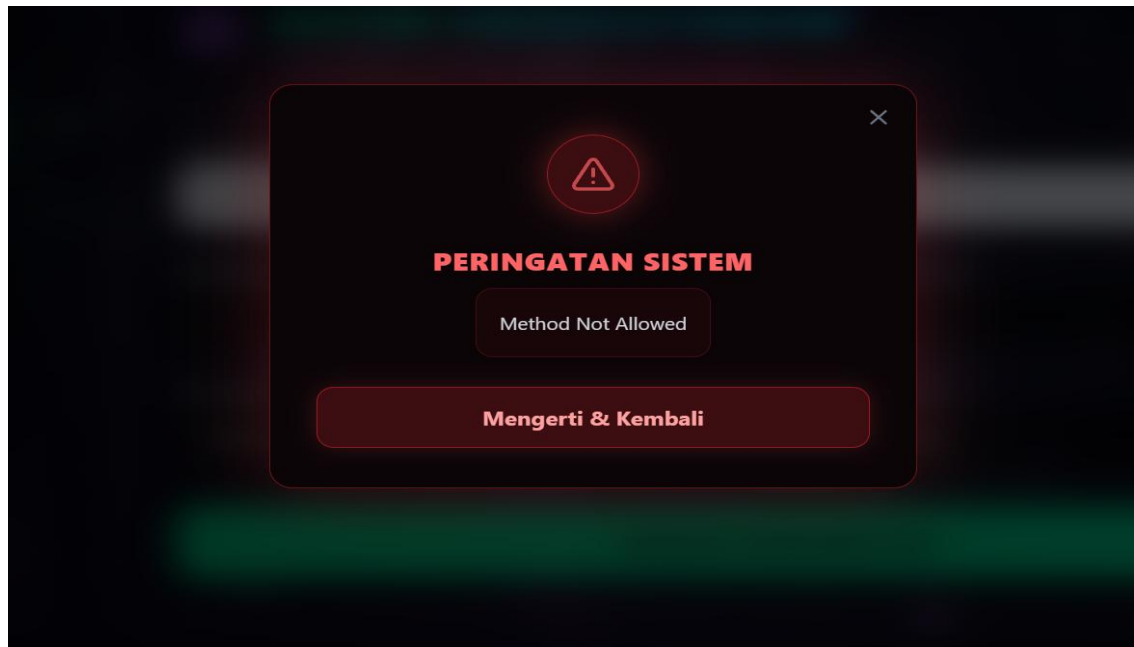
1. Skenario Normal (Alur Berhasil):

- Aksi: Operator memasukkan target "Nebula Orion" pada rentang jam 13:00 - 14:00 melalui halaman Frontend Inisiasi Observasi.
- Analisis: Lapisan Router API menerima data, meneruskannya ke JadwalManager. Validasi berhasil, data di-serialize ke dalam format JSON oleh FileHandler. Frontend kemudian memperbarui Dashboard secara otomatis. Saat tombol "BIDIK OBJEK" ditekan, Visualizer 3D sukses membaca data tersebut dan me-render Sprite Additive Blending dari tekstur lokal.



2. Skenario Kesalahan 1 (Custom Exception - Pelanggaran Syarat Bisnis):

- Aksi: Operator mencoba menginisiasi proposal pengamatan "Jupiter" pada jam 13:30, di mana rentang waktu tersebut sudah dipesan oleh proposal "Nebula Orion".
- Analisis: Saat metode `tambah_jadwal()` dieksekusi, sistem mendeteksi tabrakan slot waktu. Sistem menolak operasi tersebut dan melempar custom exception `JadwalBentrokError`. API Backend mengonversi error ini menjadi respons HTTP 400 Bad Request, dan Frontend menampilkan pesan peringatan kepada pengguna tanpa merusak antarmuka.



3. Skenario Kesalahan 2 (File Handling Failure - Kegagalan I/O):

- Aksi: Disimulasikan dengan kondisi file `jadwal_observasi.json` sengaja dirusak strukturnya atau dikosongkan secara paksa dari sistem operasi.
- Analisis: Saat sistem memanggil `FileHandler.read_json()`, kegagalan parsing terdeteksi. Sistem berhasil menangkap `json.JSONDecodeError` bawaan Python dan secara aman mengembalikan senarai kosong (`[]`) alih-alih mengalami crash (berhenti total). Sistem terbukti memiliki penanganan error yang tangguh (robust).

```
PS D:\Sistem Manajemen Observatorium Astronomi dan Jadwal Pengamatan> uvicorn main_api:app
INFO: Started server process [16700]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:63777 - "OPTIONS /api/jadwal HTTP/1.1" 200 OK
INFO: 127.0.0.1:63777 - "POST /api/jadwal HTTP/1.1" 405 Method Not Allowed
INFO: 127.0.0.1:59083 - "POST /api/jadwal HTTP/1.1" 405 Method Not Allowed
█
```

BAB V PENUTUP

5.1.Kendala Selama Pengembangan

Selama proses pengembangan sistem OBSERVA, tantangan teknis utama yang dihadapi meliputi:

1. Integrasi CORS (Cross-Origin Resource Sharing): Menghubungkan Frontend Next.js (port 3000) dengan Backend FastAPI (port 8000) memicu pemblokiran keamanan peramban. Solusinya adalah mengonfigurasi CORSMiddleware secara ketat pada modul peladen FastAPI.
2. Rendering 3D Fotorealistik: Membuat visualisasi Nebula Orion agar tidak terlihat seperti "planet padat" biasa membutuhkan eksperimen ekstensif pada React Three Fiber. Solusi terbaik dicapai dengan menggunakan teknik proyektor matriks (Sprite) berpadu dengan metode pencampuran warna terang (Additive Blending) untuk me-render tekstur awan gas.

5.2.Evaluasi Program

Secara keseluruhan, aplikasi berhasil memenuhi semua poin indikator proyek:

- Kelebihan: Struktur sistem backend sangat modular (highly decoupled) berkat arsitektur PBO (Pola Strategy dan Enkapsulasi). Di sisi Frontend, sistem memiliki nilai jual luar biasa berkat antarmuka Ultra-Clear Glassmorphism dan interaktivitas WebGL 3D Pro Max yang sangat futuristik dan responsif.
- Kelemahan: Mengandalkan file JSON tanpa mekanisme penguncian berkas (File Locking) masih menyisakan potensi Race Condition jika beberapa operator dari terminal berbeda melakukan inisiasi pengamatan secara bersamaan.

5.3.Kemungkinan Pengembangan

Untuk menyempurnakan purwarupa (prototipe) ini di masa mendatang, pengembangan dapat difokuskan pada:

1. Thread-Safe File Handling & Migrasi SQL: Menambahkan penguncian fail (file lock) pada modul FileHandler, atau memigrasikan penyimpanan lokal JSON sepenuhnya ke basis data relasional seperti PostgreSQL.

2. WebSockets untuk Real-Time Telemetry: Mengganti HTTP Polling dengan WebSocket agar pergerakan sudut teleskop dan rotasi objek di Visualizer 3D bisa benar-benar disinkronkan secara live tanpa delay sedikitpun.

5.4.Kesimpulan

Proyek Sistem Manajemen Observatorium (OBSERVA) ini telah sukses dirampungkan dengan memenuhi keseluruhan skenario wajib. Integrasi utuh pilar-pilar Pemrograman Berorientasi Objek (Inheritance, Polymorphism, Encapsulation, Custom Exception, File Handling) dan Strategy Pattern pada backend Python berhasil diwujudkan. Kombinasi backend yang tangguh tersebut dengan teknologi antarmuka Next.js 3D telah menciptakan aplikasi kendali fasilitas astronomi yang modern, elegan, tangguh, dan sangat fungsional.

GITHUB: <https://github.com/Lutpiuuww/SISTEM-MANAJEMEN-OBSERVATORIUM-ASTRONOMI-DAN-JADWAL-PENGAMATAN.git>